

QUIRÓN — MEMORIA DEL TRABAJO FIN DE MÁSTER

Versión de cierre para revisión del autor · 10 de septiembre de 2026

Esta versión actualiza el alcance del borrador de julio tras contrastarlo con el código y las evidencias disponibles. Se distinguen implementación, pruebas registradas y trabajo futuro. Los datos marcados [COMPLETAR] requieren información del autor; el documento no debe presentarse como definitivo mientras permanezcan pendientes. El estudio inicial del entrenamiento propio se conserva como antecedente de la decisión de emplear modelos preentrenados.

UNIVERSIDAD EUROPEA DE MADRID

ESCUELA DE ARQUITECTURA, INGENIERÍA Y DISEÑO

MÁSTER EN FORMACIÓN PERMANENTE EN INTELIGENCIA ARTIFICIAL APLICADA

TRABAJO FIN DE MÁSTER

Quirón: un editor nativo con índice de código verificable y una red neuronal local para el aporte de contexto a modelos de lenguaje

Autor: Lorenzo Juan Santacreu Pascual

Dirigido por: [COMPLETAR — Nombre del director/a]

Curso 2025–2026

RESUMEN

Los asistentes de programación necesitan localizar información relevante dentro de un repositorio para responder y proponer cambios. Este trabajo presenta Quirón, un editor nativo en Rust que integra un índice de código mantenido por un worker local. Al abrir un proyecto, el sistema detecta archivos, extrae unidades sintácticas de Rust, genera fichas descriptivas y produce representaciones vectoriales para su consulta. La configuración inicial utiliza Qwen2.5-Coder-1.5B cuantizado para redactar las fichas y BGE-M3 para vectorizarlas. Qdrant almacena los vectores y Neo4j representa la pertenencia de las unidades y relaciones aproximadas de llamadas. Las fichas conservan proyecto, ruta, símbolo, líneas y hash; la recuperación comprueba su vigencia frente al archivo actual.

La aportación consiste en integrar ese recorrido con el editor, la selección y descarga de modelos compatibles y distintas conexiones con asistentes. El entrenamiento de una red propia se excluyó del alcance final por razones de recursos, plazo y prioridad de validación. Las evidencias registradas incluyen indexación incremental, aislamiento entre proyectos, cambios, borrados y uso de herramientas mediante proveedores reales y simulados. También muestran limitaciones de fidelidad en los resúmenes. La evaluación disponible acredita un prototipo funcional; no permite afirmar todavía ahorro de tokens en tareas completas, prevención de duplicados ni análisis exhaustivo de dependencias.

Palabras clave: modelos de lenguaje, recuperación aumentada, índice de código, inferencia local, editor nativo, trazabilidad.

ABSTRACT

Programming assistants need to locate relevant information within a repository to answer questions and propose changes. This work presents Quirón, a native Rust editor integrating a code index maintained by a local worker. When a project is opened, the system discovers files, extracts Rust syntax units, generates descriptive records and produces vector representations for retrieval. The initial configuration uses quantized Qwen2.5-Coder-1.5B for descriptions and BGE-M3 for embeddings. Qdrant stores the vectors, while Neo4j represents unit membership and approximate call relationships. Records retain the project, path, symbol, line range and file hash; retrieval checks their freshness against the current source file.

The contribution is the integration of this workflow with the editor, downloadable and replaceable compatible models, and multiple assistant connections. Training a proprietary worker model was excluded from the final scope because of resource constraints, the available schedule and validation priorities. Recorded evidence covers incremental indexing, isolation between projects, updates, deletions and tool interactions through real and simulated providers. It also exposes limitations in the fidelity of generated descriptions. The available evaluation supports a functional prototype; it does not establish token savings for complete tasks, duplicate prevention or exhaustive dependency analysis.

Keywords: language models, retrieval augmentation, code indexing, local inference, native editor, traceability.

TABLA RESUMEN

Campo	Datos
Nombre y apellidos	Lorenzo Juan Santacreu Pascual
Título	Quirón: un editor nativo con índice de código verificable y una red neuronal local para el aporte de contexto a modelos de lenguaje
Director/a	[COMPLETAR]
Colaboración con empresa	No
Producto implementado	Sí; prototipo de editor e índice local
Investigación o innovación	Ingeniería aplicada y estudio de alternativas de modelos
Objetivo general	Integrar un índice consultable con fuentes comprobables y un worker local configurable dentro de un editor nativo

Tabla 1. TABLA RESUMEN

Capítulo 1. RESUMEN DEL PROYECTO

1.1 Contexto y justificación

Trabajar sobre proyectos grandes exige conocer qué contiene cada archivo y dónde se implementa una responsabilidad. La experiencia del autor con asistentes de programación motivó la búsqueda de un mapa persistente que facilitase localizar código existente y seleccionar contexto. Las duplicaciones, las interpretaciones equivocadas y el consumo de contexto son problemas que motivan el diseño; su reducción requiere una evaluación específica y no se deduce de disponer de un índice.

Quirón separa el trabajo continuo de descripción y búsqueda del razonamiento solicitado al asistente. El procesamiento del índice se realiza en local. El asistente puede ejecutarse mediante un proveedor remoto o un servidor compatible seleccionado por el usuario. La aplicación mantiene una relación explícita entre la ficha recuperada y el archivo que debe leerse para comprobarla.

1.2 Planteamiento del problema

La pregunta de implementación es: **¿puede un editor mantener un mapa consultable de los archivos y las lógicas de un proyecto, mediante un worker local, y entregar al asistente referencias cuya vigencia se compruebe contra el código?** La pregunta experimental adicional es si ese mapa mejora la localización y reduce el contexto necesario manteniendo la calidad de las respuestas. Ambas preguntas requieren evidencias distintas: integración funcional para la primera y comparación entre métodos para la segunda.

1.3 Objetivos del proyecto

El alcance final comprende editor, guardia de rutas, indexación incremental, fichas generadas mediante un modelo preentrenado, embeddings, proyecciones y recuperación conectada al chat. La descarga y selección de modelos compatibles permite adaptar el worker al dispositivo. La evolución de los objetivos iniciales se documenta en el Capítulo 3 y en §4.2.8.

1.4 Resultados obtenidos

Las pruebas guardadas del 5 de septiembre muestran el funcionamiento del worker Qwen/BGE-M3 con Qdrant y Neo4j sobre dos proyectos sintéticos: apertura, consultas, cambios, retirada de unidades y exclusión de archivos sensibles y enlaces. Hay evidencias de interacción del editor con Claude y Codex, así como pruebas con un servidor compatible simulado. Los resultados negativos del modelo se conservaron para estudiar

errores de contenido y truncamiento.

La auditoría del 10 de septiembre encontró discrepancias entre la memoria, el código y el paquete existente. Esta versión recoge los límites de integridad del ledger, de resolución de llamadas y de reconstrucción del índice de código. El apartado de evaluación separa estos hallazgos de las pruebas anteriores.

1.5 Estructura de la memoria

El Capítulo 2 presenta los antecedentes; el 3 fija objetivos y cambios de alcance; el 4 describe planificación, implementación y evaluación; el 5 discute límites; el 6 presenta conclusiones; el 7 recoge trabajo futuro; el 8 contiene referencias y el 9 identifica anexos y evidencias.

Capítulo 2. ANTECEDENTES / ESTADO DEL ARTE

2.1 Recuperación de contexto en repositorios

Existen trabajos previos que recuperan información del repositorio para asistir a modelos de código. RepoCoder combina recuperación por similitud y generación iterativa [25]. GraphCoder utiliza grafos de contexto de código para recuperar fragmentos [26]. RepoGraph ofrece una estructura de repositorio que sirve de apoyo a agentes de ingeniería de software [27]. Estos antecedentes impiden atribuir a Quirón la invención de la recuperación de código mediante grafos.

La aportación que se estudia aquí es la integración de un worker local configurable con un editor nativo, proyecciones incrementales y comprobación de vigencia de las fichas. No se presenta una comparación experimental que establezca superioridad frente a los sistemas anteriores. La matriz de diferencias debe interpretarse como descripción del alcance de Quirón, no como clasificación exhaustiva de productos.

Aspecto	Aportación desarrollada en Quirón
Localización	Consulta de fichas con ruta, símbolo, líneas y hash
Mantenimiento	Monitor incremental del proyecto y retirada de unidades desaparecidas
Modelos	Generador local sustituible y embeddings independientes
Interfaz	Editor nativo con estado del worker, proveedores y fuentes recuperadas
Dependencias	Pertenencia y aproximación de llamadas Rust; cobertura incompleta
Evaluación	Integración y casos exploratorios; falta comparación de tareas completas

Tabla 2. 2.1 Recuperación de contexto en repositorios

2.2 Representación, generación y almacenes

Un generador de texto y un modelo de embeddings cumplen tareas distintas. BGE-M3 [19][29] proporciona las representaciones utilizadas por esta implementación. Qwen2.5-Coder [23] genera descripciones breves. La calidad de estas descripciones puede influir en la recuperación, pero el tamaño del generador no garantiza una mejora en todas las tareas.

La literatura sobre InfoNCE [13], last-token pooling [14][15] y Matryoshka [16] sirve de base para explorar modelos de representación. El uso de un estado del último token no acredita por sí solo calidad de embeddings. Las representaciones truncables requieren entrenamiento apropiado; dos modelos con vectores de la misma dimensión no comparten necesariamente un espacio semántico. En esta versión no se entrena una representación propia ni se demuestra que los vectores de BGE-M3 puedan truncarse conservando calidad.

Qdrant [17] almacena los vectores y permite restringir candidatos mediante filtros de carga útil. Neo4j [18] almacena nodos y relaciones consultables. La identidad de las unidades permite vincular las dos proyecciones. La existencia de un grafo no implica que sus relaciones resuelvan todos los tipos, importaciones o llamadas del programa.

2.3 Estudio de alternativas para la red obrera

El estudio inicial revisó MLA [1][2], atención dispersa [3], mecanismos lineales [4–7], mezcla de expertos [8][9], destilación [10–12c] y cuantización [20][21]. Su resultado se conserva en el Anexo A como exploración de alternativas. Ninguno de esos mecanismos fue implementado como un nuevo backbone entrenado por el autor.

La recurrencia de estado fijo puede tener coste constante por paso respecto a la longitud del contexto, para dimensiones fijas. Esta propiedad no se extiende al modelo completo si se añaden capas de atención completa sobre todo el historial. La exportación y el rendimiento de una arquitectura híbrida en ONNX/Rust se dejaron como cuestiones por medir.

Sequence-Level KD [11] permite utilizar respuestas textuales de un profesor. QLoRA [24] muestra que es posible ajustar modelos preentrenados reduciendo el consumo de memoria. La decisión de no entrenar en esta versión es de alcance y viabilidad; no supone que cualquier ajuste resulte imposible con el equipo disponible.

2.4 Justificación del proyecto

Se busca facilitar la navegación por responsabilidades del código, conservar referencias comprobables y adaptar la inferencia al equipo del usuario. El ahorro económico y la prevención de lógica duplicada permanecen como hipótesis de aplicación. El cómputo local tiene costes de descarga, almacenamiento, memoria, energía y mantenimiento; disponer del hardware no los elimina. Las suscripciones y las conexiones API son vías de acceso diferentes con condiciones y límites propios.

Capítulo 3. OBJETIVOS

3.1 Objetivo general y evolución del alcance

El objetivo final es **construir y evaluar un prototipo de editor nativo que mantenga un índice de código mediante un worker local configurable y aporte al asistente referencias comprobables contra los archivos del proyecto**. La versión inicial incluía entrenamiento propio, historial completo de cambios y análisis amplio de dependencias. El autor decidió priorizar el recorrido funcional con modelos preentrenados. Se conserva la numeración de objetivos para hacer visible esa evolución; no se presenta una reformulación como cumplimiento retroactivo del objetivo inicial.

3.2 Objetivos específicos y estado

Objetivo	Planteamiento inicial	Alcance y estado al cierre
OE1	Registro inmutable y dos proyecciones reconstruibles desde él	Registro encadenado v2 con hash de todos los campos, transacciones y anclaje compatible; herramientas de replay de eventos implementadas; reconstrucción completa del índice de código pendiente
OE2	Editor con confinamiento al proyecto	Editor y guardia de rutas implementados, con pruebas; quedan límites ante carreras del sistema de archivos
OE3	Indexador incremental de Archivo, Lógica y Cambio	Archivo y lógica Rust operativos; tipo Cambio definido sin historial completo integrado
OE4	Grafo de dependencias resuelto por análisis estático	Pertenencia y llamadas aproximadas Rust; resolución de tipos, imports y dependencias completas pendientes
OE5	Vector, expansión, reranker y presupuesto de contexto	Filtro en la búsqueda de código y expansión limitada implementados; reranker y presupuesto global pendientes en ese recorrido
OE6	Estudiar mecanismos para una red propia exportable	Estudio de alternativas documentado; no se acredita un nuevo backbone ni su exportación

Objetivo	Planteamiento inicial	Alcance y estado al cierre
OE7	Entrenar por destilación y desplegar una red propia	Entrenamiento excluido; objetivo reformulado a integrar, descargar y seleccionar modelos preentrenados compatibles
OE8	Medir ahorro, recuperación, aislamiento y fidelidad	Evidencias funcionales y evaluación exploratoria; la hipótesis de ahorro en tareas completas no está demostrada

Tabla 3. 3.2 Objetivos específicos y estado

3.3 Beneficios previstos y límites de la aportación

El índice facilita localizar código que podría reutilizarse y revisar la procedencia de la información recuperada. Un hash coincidente acredita vigencia del archivo, no la corrección de su resumen. Quirón no dispone de un detector automático de lógica duplicada conectado al editor ni de resúmenes consolidados de responsabilidades por carpeta. Esas extensiones no forman parte de los resultados de esta versión.

Capítulo 4. DESARROLLO DEL PROYECTO

4.1 Planificación

La planificación inicial situaba infraestructura y editor antes del entrenamiento. Durante septiembre se sustituyó el entrenamiento propio por la integración de modelos preentrenados y se priorizó el cierre funcional. Los periodos de febrero a julio proceden del borrador del autor y requieren confirmación para calcular horas; el historial Git y las evidencias de septiembre permiten identificar las revisiones recientes.

Etapa	Trabajo y evolución
Febrero–junio, según planificación inicial	Infraestructura de servicios y editor nativo
Junio–julio, según planificación inicial	Memoria de eventos, diagnóstico de recuperación y estudio matemático
Septiembre, evidencia del repositorio	Worker Qwen/BGE-M3, integración de almacenes, proveedores, interfaz y candidato de instalación
Revisión del 10 de septiembre	Auditoría entre código y memoria, delimitación del alcance y preparación de evidencias de cierre

Tabla 4. 4.1 Planificación

La entrega por repositorio fue indicada por el autor como vía acordada con el tutor. URL: **[COMPLETAR — repositorio de entrega]**. Un identificador de commit identifica una revisión; las modificaciones sin commit y los artefactos generados deben acompañarse de un inventario de hashes.

4.2 Solución, metodología y herramientas

4.2.1 Memoria de eventos e índice de código

El cerebro utiliza Sled como almacenamiento local; el nombre histórico `storage/rocks.rs` no significa que se utilice RocksDB. Los eventos se encadenan mediante Blake3. Existen envolturas de memoria con estados y relaciones de sustitución o retractación, y herramientas de reconstrucción de proyecciones de eventos.

El índice de código nuevo mantiene su manifiesto y la caché de fichas/vectores en Sled, y obtiene el contenido de los archivos. Confirma un archivo después de recibir el acuse de Qdrant y Neo4j. Comprueba periódicamente la presencia de unidades para reparar determinadas pérdidas mediante la caché. Este mecanismo no equivale a reconstruir íntegramente el índice desde el ledger: faltan eventos suficientes para reproducir todo su historial.

La auditoría inicial encontró campos sin cubrir y escrituras separadas. La corrección v2 calcula el hash sobre la serialización completa del evento, salvo su propio hash, con separación de dominio y longitudes delimitadas. Evento, índices, versión, anclaje y cabecera se escriben en una transacción Sled que solicita persistencia antes de confirmar. Los escritores comparten un bloqueo; los identificadores duplicados se rechazan. La verificación propaga los registros ilegibles en lugar de omitirlos.

Los eventos históricos conservan sus bytes y su algoritmo. El primer evento v2 incorpora un anclaje del contenido completo observado al migrar; no demuestra su originalidad antes de ese momento. La prueba de interrupción termina un proceso después de confirmar lotes y verifica su recuperación íntegra. No se ha simulado pérdida de alimentación. Tampoco se acredita resistencia frente a un actor capaz de reescribir toda la base y su cabecera, al no existir una raíz de confianza externa. Las envolturas derivadas de memoria siguen fuera de la transacción autoritativa.

4.2.2 Identidad y recorrido del proyecto

La identidad se conserva en `.quiron/project.id`; la aplicación incluye migración desde el estado histórico `.llore`. El worker exige una raíz válida y una identidad coincidente. Una copia con la misma identidad no puede monitorizarse simultáneamente como otro proyecto independiente. La separación se verifica con proyectos de identidades distintas.

El recorrido excluye enlaces, nombres sensibles y directorios generados. Cuando la raíz es un repositorio Git, respeta el conjunto de archivos seguido/no ignorado por Git. Sus límites son 512 KiB por archivo, 10 000 archivos y 64 MiB de texto por barrido. Las reglas se basan en rutas y nombres; no garantizan detectar credenciales incrustadas dentro de cualquier archivo de código.

4.2.3 Archivos, lógicas y fichas

Se genera una unidad de archivo para las extensiones admitidas. Tree-sitter añade unidades de lógica Rust: funciones, métodos, estructuras, enumeraciones y traits con firma y líneas. La cobertura no incluye toda construcción del lenguaje, y los otros lenguajes tienen por ahora ficha de archivo.

El worker v5 proporciona al generador hasta 6 000 caracteres de la unidad, numerados por línea, y definiciones Rust de constantes referenciadas cuando caben en el contexto acotado. Estas definiciones participan en la huella de caché. Las entradas recortadas se marcan como parciales. La respuesta pide un propósito en una frase completa, contexto no resuelto y entre una y tres líneas de evidencia. El programa copia los fragmentos citados desde el código y convierte sus líneas a posiciones del archivo.

Se rechazan frases incompletas, referencias inexistentes, cifras ausentes del contexto y determinadas menciones de lenguajes sin respaldo en la entrada, además de salidas inválidas o que repiten instrucciones. En esos casos se guarda una ficha estructural con el motivo y `summary_origin=parser`; las aceptadas llevan `model`. Los errores de conexión siguen siendo reintentables. Las reglas y las citas verifican restricciones concretas, no toda afirmación semántica de la descripción.

BGE-M3 vectoriza ruta, símbolo, firma y descripción. La ficha mantiene la relación con el archivo sin copiar todo su código al texto del vector. El extractor antiguo calcula una huella del cuerpo normalizado por espacios; no existe un comparador automático de duplicados conectado a esa huella, y el worker nuevo no la incorpora como criterio de comparación entre funciones.

4.2.4 Grafo y recuperación

Neo4j contiene `CodeProject`, `CodeUnit`, `HAS_UNIT`, `DEFINED_IN` y `CALLS`. Las llamadas se extraen sintácticamente y se enlazan mediante reglas por nombre y archivo. Se omiten casos ambiguos según esas reglas, pero una coincidencia única de nombre no prueba la resolución semántica del destino: se pierden determinados prefijos de módulo y no se resuelve el tipo del receptor. No se ofrecen garantías de análisis de impacto completo.

La búsqueda de código aplica dentro de Qdrant filtros de proyecto y modelos. Revalida ruta y hash antes de devolver cada acierto. Añade un vecino por llamadas cuando puede obtenerlo del grafo. El límite solicitado restringe los aciertos vectoriales y la expansión puede aumentar el número final. No se ejecuta un reranker de fichas ni se aplica un presupuesto global medido con el tokenizador del asistente a ese conjunto.

La consulta de vecinos filtra explícitamente proyecto y versiones de modelos en la semilla y el vecino, y conserva la ficha JSON y su indicador de entrada parcial. Estas correcciones se comprobaron introduciendo aristas entre proyectos de contenido idéntico y nodos de versiones anteriores. La memoria de eventos utiliza otro recorrido: parte del filtrado por proyecto sigue ocurriendo después de buscar. Las garantías verificadas del índice nuevo no deben atribuirse automáticamente a todos los endpoints.

4.2.5 Editor y herramientas

El editor utiliza `winit`, `tiny-skia`, `softbuffer`, `taffy` y `cosmic-text`. Las tipografías se incluyen en el producto. Permite abrir proyectos, editar archivos y consultar el chat; muestra el estado del worker y las fuentes recibidas. La paleta Agentes integra selección de proveedores y modelos del worker. El manual describe las operaciones de la interfaz.

Las herramientas ofrecidas al asistente en `chat_tools.rs` son lectura de archivos, listado y búsqueda textual. Se ejecutan mediante la guardia del proyecto y con límites de salida. Su existencia no implica edición autónoma completa ni que cada respuesta consulte todos los archivos pertinentes. Persisten carreras posibles entre inspeccionar una ruta y abrirla; el modelo de uso evaluado es un escritorio local.

4.2.6 Proveedores y suscripciones

El gateway admite conexiones con CLI de Claude, sesión de Codex, servidores compatibles con OpenAI y Ollama. Una suscripción no proporciona un identificador universal de acceso: cada adaptador tiene su propio mecanismo y restricciones. Claude y la conexión nueva `codex_cli` utilizan sus CLI autenticadas. El adaptador histórico `codex_direct` se conserva para configuraciones anteriores. El selector reúne los alias de Claude (Sonnet, Opus y Haiku) y el catálogo de Codex, que puede actualizarse. Cada conversación transmite su proveedor y modelo sin cambiar la configuración del servicio; la elección de Claude o Codex no detiene el worker. En Codex también se transmite el esfuerzo de razonamiento, sin forzar GPT-5.5 cuando hay herramientas. Las conexiones compatibles pueden requerir una clave de API.

Codex CLI se ejecuta en un directorio temporal, sin configuración personal de ejecución y con las herramientas nativas de acceso a archivos, comandos y servicios desactivadas, con un contrato JSON cuyas llamadas ejecuta la guardia del editor. La prueba de transporte con CLI 0.153.0 comprueba el modelo, el esfuerzo y esa restricción incluso cuando no se obtiene un catálogo remoto. En ese caso la CLI puede anunciar su herramienta de preguntas del modo Plan, que no da acceso al proyecto ni permite entrada interactiva en esta ejecución. Es una conversación propia de Quirón, sin heredar el hilo del IDE.

Las evidencias incluyen proveedores reales y un servidor simulado que comprueba transporte de herramientas. Una prueba con ese servidor verifica integración, no calidad de un modelo comercial. La configuración y el proveedor empleado deben registrarse en cada experimento. Los resultados históricos no garantizan disponibilidad futura ni uso ilimitado de una suscripción.

4.2.7 Worker descargable y adaptación al dispositivo

La configuración inicial emplea Qwen2.5-Coder-1.5B-Instruct Q4_K_M mediante `llama.cpp` y BGE-M3 en CPU. El catálogo de `setup-worker.py` permite descargar modelos con revisión y SHA-256 fijos; la paleta permite seleccionar un GGUF compatible. El cambio de modelo está separado del editor y los almacenes. Las recomendaciones de memoria del catálogo son orientativas; no equivalen a benchmarks de todos sus modelos.

Un dispositivo con más recursos puede evaluar un generador de mayor capacidad manteniendo el mismo recorrido. Una ficha más precisa puede aportar mejores datos para recuperar contexto, pero hay que medir calidad, latencia y memoria. El generador no sustituye automáticamente al modelo de embeddings y no se afirma que el mayor modelo siempre mejore el sistema. La identificación actual del generador utiliza su nombre de archivo; sustituir pesos con el mismo nombre requiere especial cuidado para invalidar correctamente la caché.

4.2.8 Selección de modelos preentrenados y delimitación del entrenamiento propio

El autor declara disponer de dos estaciones de trabajo adicionales accesibles mediante SSH y de recursos que incluyen dos NVIDIA RTX 3090. Esta disponibilidad permite plantear inferencia y ajustes acotados de modelos existentes. No se ha realizado en esta revisión un inventario remoto ni una medición de entrenamiento en esas estaciones.

Entrenar desde cero y ajustar un modelo preentrenado son tareas diferentes. Como referencia de escala, el informe de Qwen2.5-Coder describe entrenamiento continuado sobre más de 5,5 billones de tokens [23]; esa cifra no es un requisito mínimo universal. Un ajuste mediante técnicas como QLoRA reduce las exigencias de memoria [24]. La presencia de dos GPU no constituye automáticamente un único espacio de memoria; el reparto del modelo y la comunicación requieren configuración específica.

El planteamiento inicial contemplaba destilación de una red propia. El autor decidió excluir ese entrenamiento del alcance final para concentrar el trabajo en la indexación, la recuperación y su validación. Pesaron la preparación de un corpus propio revisado, el coste de iteración y el plazo disponible. No se dispone de una comparación que demuestre que un modelo propio necesariamente rendiría peor, ni de un presupuesto medido de ese entrenamiento descartado.

Quirón conserva la función prevista del worker mediante modelos preentrenados sustituibles: describir archivos y lógicas, producir datos para la vectorización y mantener un mapa consultable. El código propio integra y controla ese recorrido. La selección por dispositivo permite evolucionar el componente aprendido sin atribuir al autor el entrenamiento de los pesos descargados. Un ajuste futuro quedaría condicionado a demostrar una mejora frente a esta base.

4.2.9 Metodología y reproducibilidad

Se distinguen cuatro niveles: afirmación del diseño, implementación inspeccionada, prueba funcional registrada y resultado experimental comparativo. Las salidas JSON y las capturas sirven como evidencia de recorridos concretos. Una prueba que pasa no acredita comportamientos que no ejercita. Las evaluaciones deben registrar revisión del código,

modelos, corpus, preguntas, métricas y limitaciones; los errores se conservan.

4.3 Recursos

Recurso	Uso y alcance de la evidencia
Portátil RTX 3060 de 6 GB y 15 GiB de RAM utilizable	Equipo de las pruebas locales registradas
Dos estaciones adicionales y dos RTX 3090, declaradas por el autor	Recursos disponibles; no se atribuyen resultados de entrenamiento
Rust, Sled, Qdrant, Neo4j y servicios de usuario	Editor, almacenamiento y procesos locales
Qwen, llama.cpp y BGE-M3	Inferencia de fichas y embeddings
CLI y endpoints configurables	Acceso a asistentes; condiciones dependientes del proveedor

Tabla 5. 4.3 Recursos

El coste inicial incluye descargas: aproximadamente 1,15 GB para el runtime y modelo inicial del worker, además de BGE-M3 y las imágenes de los almacenes. No se descargó un corpus para entrenar una red propia. La observación de VRAM del servidor Qwen fue de unos 1 130 MiB en la prueba registrada; no representa su pico máximo ni el consumo de todas las configuraciones.

4.4 Presupuesto

Concepto	Importe o base disponible
Dedicación del autor	[COMPLETAR — horas y criterio de valoración]
Hardware	[COMPLETAR — coste imputado o amortización; distinguir equipos disponibles y usados]
Suscripciones y servicios	[COMPLETAR — gastos reales por periodo]
Electricidad y almacenamiento	[COMPLETAR — estimación y método, si se incluyen]
Entrenamiento propio	No realizado; no se declara un gasto ni un ahorro medido

Tabla 6. 4.4 Presupuesto

No se presenta un coste total sin los datos anteriores. La disponibilidad de software y pesos descargables no elimina las condiciones de licencia ni los costes operativos. La revisión de licencias de distribución debe acompañar al paquete.

4.5 Viabilidad y despliegue

Existe un candidato de instalación por usuario para Linux x86_64 que prepara configuración privada, servicios y lanzador. Requiere Python, Docker, systemd y acceso a las descargas. No es un paquete offline ni una distribución certificada para cualquier Linux. La comprobación mediante `--destdir` verifica archivos y configuración sin activar servicios; no sustituye una prueba de arranque en equipo limpio.

La comprobación de bibliotecas del editor compilado el 10 de septiembre en una base Debian 13 con glibc 2.41 falló por el símbolo `acosf@GLIBC_2.43`. Se utilizó una imagen local aislada, sin red, montando únicamente el binario en lectura. Es una prueba negativa del primer candidato, no una instalación completa. La entrega posterior se compila en Ubuntu 24.04 y limita sus símbolos requeridos a glibc 2.39; el detalle de sus comprobaciones se conserva en el informe de estabilización. La base Ubuntu 22.04 se descartó al comprobar que el archivo estático fijado de ONNX Runtime utiliza funciones C23 ausentes en su glibc. Las pruebas de bibliotecas y staging no sustituyen una instalación limpia completa.

El cerebro mantiene los almacenes y el worker. El cierre de la ventana permite continuar trabajo en segundo plano; el apagado por inactividad considera peticiones y barridos activos. Las unidades limitan memoria del host y la caché de prompts. Estos límites se añadieron después de una incidencia de presión de RAM documentada el 5 de septiembre.

4.6 Resultados y evaluación

4.6.1 Evidencias funcionales registradas

Evidencia del 5 de septiembre	Qué acredita	Límite
<code>worker-integracion.json</code>	Dos proyectos sintéticos, consulta, cambios, borrados, exclusión de secretos/enlaces y caché	No mide precisión en proyectos arbitrarios
<code>worker-verificacion.json</code>	299 tests de bibliotecas del editor, 175 del cerebro, 22 del gateway y 8 de entrega; 504 en total, con 2 ignorados	Revisión de ese día; no equivale a repetirlos sobre el código actual
<code>agentes/resumen.json</code>	Rondas de interfaz con Claude, Codex, servidor local y escenarios compatibles simulados	Distinguir inferencia real de transporte probado mediante simulación
<code>consulta-codigo.json</code>	Consultas con fuentes, control de vigencia y uso del chat	Muestra pequeña; incluye respuestas incompletas

Evidencia del 5 de septiembre	Qué acredita	Límite
worker-ficha-v2-evaluacion.json, worker-caso-real.json, worker-english-evaluacion.json	Errores de contenido, repetición de instrucciones y truncamiento	No constituyen una tasa representativa de error

Tabla 7. 4.6.1 Evidencias funcionales registradas

La verificación histórica del ledger registró 5 057 eventos y una cadena válida bajo el algoritmo implementado. El significado de esa validación queda limitado por §4.2.1. Las cifras del borrador de julio —5 028 eventos y 2,4 ms de fotograma— no se extrapolan a la interfaz ni al conjunto de datos actuales.

El 10 de septiembre se repitieron las bibliotecas del editor (321 pruebas aprobadas y 2 ignoradas), el cerebro con full (177), el gateway (26) y las pruebas de entrega (9): 533 aprobadas en total. Los comandos y resúmenes se conservan en `verificacion.json`. Las pruebas de entrega usan Docker simulado; no acreditan una instalación limpia con bases reales. El primer intento del editor no pudo abrir los sockets locales de prueba dentro del sandbox; la ejecución con acceso a localhost terminó sin fallos.

Se repitió además la integración real sobre dos proyectos temporales: autenticación, consultas, cambios, borrados, proyecciones y ausencia de inferencia sin cambios. En una ventana del editor se comprobó abrir, editar, guardar, deshacer y rehacer con estados intermedios, cerrar y reabrir conservando contenido e identidad, y actualizar el índice. El arnés se corrigió porque suponía que Ctrl+End iba al final del documento; la implementación lo interpreta por línea. Estas pruebas se hicieron en el equipo de desarrollo y sus evidencias se conservan en `worker-integracion.json`, `gui/gui-edit.json` y `compatibilidad.json` del cierre del día 10.

4.6.2 Evaluación exploratoria de cierre

El protocolo de cierre utiliza preguntas de localización con símbolos esperados identificados en el código antes de consultar el índice. Mide si el objetivo aparece entre los primeros cinco resultados vectoriales y en el conjunto ampliado, y registra el rango de la primera coincidencia. Una coincidencia de símbolo es un indicador de localización; no demuestra que la respuesta explique correctamente su comportamiento.

Se compara además el volumen de las fichas recuperadas con el texto de un conjunto explícito de archivos de código bajo el mismo tokenizador. Esta comparación es una medida del material de contexto, no del coste de una tarea completa ni de la facturación de un proveedor. El conjunto de preguntas es pequeño, elaborado durante el desarrollo y sin separación de entrenamiento/evaluación del sistema; sus resultados son exploratorios.

La ejecución del 10 de septiembre se conserva en docs/evidencias/2026-09-10/localizacion.json, junto con el protocolo docs/evaluacion/localizacion-v1.json y el script scripts/evaluate-index.py. El conjunto de referencia contiene 140 archivos Rust versionados bajo src/, de hasta 512 KiB cada uno. La búsqueda se hizo sobre el índice del proyecto completo, con los modelos ya cargados.

Medida	Resultado observado
Preguntas con objetivo entre los cinco resultados directos (Hit@5)	10 de 10
Media del inverso del rango del objetivo, hasta cinco (MRR@5)	0,75
Preguntas con objetivo tras la expansión por llamadas	10 de 10; no aumenta los aciertos en esta muestra
Mediana de duración de la petición de búsqueda	0,111 s
Fichas devueltas, contando apariciones entre preguntas	75; todas coinciden con el proyecto solicitado y con el hash del archivo actual
Texto de referencia de los 140 archivos	420 881 tokens
Mediana del conjunto de fichas por pregunta, incluida expansión	2 491,5 tokens; 0,592 % del volumen de referencia

Tabla 8. 4.6.2 Evaluación exploratoria de cierre

Hit@5 cuenta preguntas cuyo símbolo esperado aparece en los cinco primeros resultados; MRR@5 premia que aparezca antes. Se empleó el tokenizador de Qwen2.5-Coder-1.5B-Instruct Q4_K_M mediante /tokenize, sin tokens especiales. El informe conserva las preguntas, objetivos, resultados completos, hashes del corpus y tiempos. No se detectaron cambios de esos archivos durante la evaluación.

El volumen menor de las fichas no se presenta como un porcentaje de ahorro económico: enviar todo el código es solo una referencia de tamaño. Las preguntas proceden del desarrollo, no de un conjunto independiente; tampoco se puntuó la fidelidad de cada resumen ni se comparó el éxito de una tarea completa. Los diez aciertos permiten repetir una prueba de localización, pero no estimar precisión general ni atribuir una mejora al grafo.

Una inspección posterior de las fichas de esos objetivos encontró errores concretos: collect_calls se describe como análisis de TypeScript aunque su integración utiliza Rust; read_file menciona 1 000 caracteres cuando la constante es 24 000; y stable_id sitúa al final dos bytes cero que el código intercala como separadores. También hay descripciones que terminan a mitad de frase. Los ejemplos y fragmentos de

contraste se conservan en fidelidad-observaciones.json. Esta inspección asistida forma parte de la auditoría de desarrollo, no de una valoración independiente ni de una tasa de error representativa. Confirma que la firma y el código deben prevalecer sobre el texto generado.

4.6.3 Validación pendiente

Quedan pendientes una evaluación independiente de fidelidad de fichas, tareas completas con y sin índice, una referencia de recuperación léxica o sintáctica, repetición con varios repositorios y una instalación limpia completa. El benchmark `ledger_bench.rs` mide lectura/escritura del registro; no mide ahorro de contexto, detección de duplicados ni durabilidad de cada evento frente a pérdida de alimentación.

4.6.4 Estabilización posterior a la auditoría

Las correcciones del mismo día se conservan en `docs/ESTABILIZACION_2026-09-10.md` y `docs/evidencias/2026-09-10-estabilidad/`. El cerebro pasó 184 pruebas con full, incluidas alteraciones de campos antes omitidos, colisiones por concatenación, rechazo de lotes duplicados sin escrituras parciales, cuatro escritores concurrentes, registros corruptos, transición histórica y recuperación tras terminar un proceso. Son siete pruebas más que en la ejecución inicial.

La herramienta offline `ledger_admin` se ensayó sobre una copia antes de aplicarla al servicio. Conservó byte a byte los 5 284 eventos existentes y añadió un evento de anclaje: 5 285 eventos válidos, de ellos uno v2, en la comprobación posterior. Se guardaron una copia privada de los datos y el binario anterior. El anclaje protege el contenido observado en esa transición, con las limitaciones de §4.2.1.

La integración Qwen/BGE-M3/Qdrant/Neo4j aprobó 13 comprobaciones, incluidas actualización, borrado, caché y filtros de vecinos entre proyectos y modelos. Una muestra de ocho funciones comprendió los tres ejemplos problemáticos de la auditoría y cinco funciones sintéticas adicionales. Siete descripciones fueron aceptadas y una se sustituyó por una ficha estructural al no terminar una frase completa. Las citas y hashes coincidieron con las fuentes; los tres errores anteriores no reaparecieron. Las descripciones todavía pueden omitir ramas, unidades o efectos. No se establece una tasa de fidelidad semántica ni se extrapola el Hit@5 de v4 a v5 sin repetir la evaluación.

Una ejecución intermedia fue interrumpida por el apagado por inactividad del servicio de desarrollo, que detuvo sus almacenes. Se conservó ese fallo y se repitió la integración manteniendo activo el servicio durante el ensayo; el resultado final fue satisfactorio. Se eliminaron únicamente los datos temporales de esas pruebas.

El paquete recompilado limita sus requisitos a GLIBC_2.39 y sus cinco binarios resuelven bibliotecas en las bases Ubuntu 24.04 y Debian 13 probadas. En Ubuntu se abrió una ventana Xvfb a 1280 × 720 y se cerró normalmente; también se comprobó el instalador en staging. Esa prueba descubrió bibliotecas X11 cargadas dinámicamente que ldd no detectaba: se añadieron a la imagen y al control previo del instalador. No se certifica el ciclo completo de Docker/systemd en una VM limpia. El editor pasó de nuevo 321 pruebas, con dos ignoradas, y la entrega nueve; junto con las 184 del cerebro suman 514 aprobadas en esta estabilización. El gateway conserva las 26 pruebas aprobadas en la revisión inicial.

4.6.5 Conexión de Codex y selección de modelos

La revisión siguiente sustituyó el catálogo fijo del selector por los modelos visibles de la sesión de Codex CLI 0.153.0: GPT-6 Astra, GPT-5.6 Sol, Terra y Luna, GPT-5.5 y GPT-5.3 Codex Spark. Se eliminó la sustitución por GPT-5.5 en las conversaciones con herramientas y se añadió un selector de esfuerzo cuyos niveles dependen del modelo. El modelo y el esfuerzo se conservan por proyecto y proveedor; el control de esfuerzo se limita al adaptador nuevo que lo transporta.

La prueba real se realizó en un proyecto sintético con una función Rust de cuatro líneas que duplica un entero mediante `checked_mul`. El worker produjo dos fichas, de archivo y función, sin recurrir al analizador estructural. El chat recuperó ambas fichas con sus referencias y hash; Astra solicitó leer el archivo y respondió `Some(14)` para la entrada siete y `None` para el máximo `u32`. Las dos llamadas registraron `gpt-6-astra` con esfuerzo `xhigh`. El editor midió 49,0 segundos; el archivo permaneció intacto y la ventana cerró normalmente. La evidencia identifica el binario final y se conserva en `docs/evidencias/2026-09-10-codex/gui-sintetica/`. Es una prueba del recorrido integrado, no una evaluación de calidad o rendimiento en repositorios grandes.

Una ronda inicial respondió, pero su arnés falló al guardar la evidencia por un argumento duplicado. Se corrigió y se repitió la ronda completa con el archivo ya indexado. Los selectores se revisaron visualmente y las pruebas automatizadas sumaron 544 aprobadas: 323 del editor, 184 del cerebro, 28 del gateway y nueve de entrega, con dos pruebas del editor ignoradas. Se repitieron las comprobaciones de binarios, ventana y staging descritas en §4.6.4 para esta compilación. El informe `docs/CODEX_EN_QUIRON_2026-09-10.md` distingue los resultados reales del ensayo de transporte con un servidor simulado y mantiene los límites de validación pendientes.

4.6.6 Coexistencia de Claude y Codex

La primera incorporación de Codex filtraba el selector por el proveedor activo y ocultaba las opciones de Anthropic. La corrección reúne Claude Sonnet, Opus y Haiku —alias resueltos por su CLI— y los modelos visibles de Codex. Se mantiene Claude Sonnet como

predeterminado del servicio. Cada conversación transporta su elección mediante provider, sin reescribir la configuración privada ni reiniciar el cerebro o el worker. El gateway restringe ese campo a las dos CLI y rechaza su uso para cambiar la ruta del worker; las peticiones anteriores conservan el proveedor configurado.

La revisión pasó 324 pruebas del editor, 184 del cerebro y 29 del gateway, y conserva las nueve pruebas de entrega de la ronda anterior: 546 aprobadas y dos ignoradas. La ejecución inicial de los compiladores fue interrumpida con SIGTERM; tras retirar temporales de las pruebas y reconstruir la caché del editor, la repetición terminó correctamente. Las evidencias se conservan en docs/evidencias/2026-09-10-proveedores/ y distinguen las pruebas de transporte simulado, las de interfaz y las de carga de los binarios.

Las dos conexiones se comprobaron desde la ventana sobre la misma función sintética. La CLI de Claude resolvió sonnet como claude-sonnet-5; Codex utilizó gpt-6-astra con esfuerzo xhigh. Cada proveedor realizó dos llamadas, recibió dos fichas del índice, solicitó leer el archivo y devolvió correctamente Some(14) y None. El editor registró 12,8 y 32,8 segundos, respectivamente, sin que estas dos rondas permitan comparar rendimiento general. Al pasar de Claude a Codex no cambiaron los PID del cerebro y del worker ni el archivo privado de configuración; el servicio mantuvo Claude Sonnet como predeterminado. Las ventanas cerraron normalmente y el código no se modificó.

Capítulo 5. DISCUSIÓN

La elección de modelos preentrenados permite aprovechar capacidades desarrolladas por terceros y concentrar el trabajo propio en el recorrido del producto. La separación entre generador y embeddings permite estudiar mejoras de cada componente. Las dos torres amplían las posibilidades futuras de experimentación; su existencia no convierte en necesario entrenar un modelo para esta entrega.

Las fichas ya permiten describir archivos y determinadas lógicas. Las responsabilidades agregadas por carpeta y la detección automática de duplicados no están implementadas. La búsqueda por similitud puede proporcionar candidatos para una revisión, pero similitud no equivale a identidad de comportamiento. El hash comprueba correspondencia con el archivo; no valida la semántica del resumen.

Las garantías deben expresarse con precisión. El nuevo índice depende de los archivos y de la caché local; el ledger no contiene todo lo necesario para reconstruirlo. Las relaciones de llamadas emplean aproximaciones; la recuperación ampliada ya aplica filtros explícitos y conserva metadatos, pero eso no convierte las aristas en un análisis semántico completo. Los resultados de aislamiento sobre dos proyectos son evidencia acotada, no una prueba formal de todas las consultas posibles.

La validación comparativa puede comenzar con un presupuesto pequeño. Contar tokens offline no requiere pagar una petición que incluya todo el repositorio, pero por sí solo tampoco demuestra calidad equivalente. Una evaluación de tareas debe contabilizar instrucciones, fichas, lecturas adicionales, historial, resultados de herramientas y respuesta. No se presenta el coste de una prueba que no se realizó como evidencia de ahorro.

La auditoría encontró además documentación de estados diferentes y un paquete anterior al código revisado. La reproducibilidad de la entrega exige que Markdown, DOCX/PDF, fuentes, binarios y evidencias indiquen su revisión y alcance. Las capturas muestran una ejecución, mientras que un paquete debe verificarse por separado.

Capítulo 6. CONCLUSIONES

6.1 Conclusiones del trabajo

Se ha construido un prototipo de editor nativo que integra un worker local, fichas de archivos y lógica Rust, búsqueda vectorial, un grafo parcial y referencias comprobables contra el código vigente. El sistema incorpora descarga y selección de modelos compatibles y distintas conexiones con asistentes. Las pruebas registradas demuestran varios recorridos completos de integración y han permitido identificar fallos de calidad y recursos.

El entrenamiento propio fue un objetivo inicial reformulado de manera explícita. La contribución implementada se encuentra en la integración, el mantenimiento incremental y el control del contexto. El estudio matemático conserva valor como exploración de alternativas, sin presentarse como una nueva arquitectura entrenada o evaluada.

La hipótesis de reducción de contexto manteniendo la calidad de tareas completas sigue abierta. Siguen pendientes la reconstrucción completa del índice desde el registro y un análisis completo de dependencias. El ledger v2 ya cubre todos los campos y confirma escrituras transaccionales; su anclaje histórico y la prueba de interrupción tienen los límites explícitos de §4.2.1. El grado de cumplimiento se recoge en el Capítulo 3 para que los resultados puedan evaluarse sin confundir el prototipo con la totalidad del diseño inicial.

6.2 Reflexión del autor

[COMPLETAR — reflexión personal del autor sobre el aprendizaje, las decisiones y las dificultades. Este apartado debe expresar su experiencia y no atribuirle conclusiones personales redactadas por terceros.]

Capítulo 7. FUTURAS LÍNEAS DE TRABAJO

- Ampliar las pruebas del ledger v2 a fallos de almacenamiento y pérdida de alimentación, y estudiar puntos de control externos si se requiere detectar la reescritura de toda la base.
- Registrar cambios de código suficientes para reconstruir las proyecciones desde evidencia histórica y probar su equivalencia lógica.
- Resolver mejor módulos, tipos y llamadas, y ampliar las pruebas de recuperación y aislamiento a más repositorios.
- Incorporar reranking y presupuesto de contexto con el tokenizador del consumidor, medidos en tareas completas.
- Comparar generadores y cuantizaciones por fidelidad, latencia y memoria antes de seleccionar una configuración superior.
- Evaluar LoRA/QLoRA o destilación especializada sobre ejemplos revisados, si aporta mejoras frente al modelo base.
- Detectar candidatos a duplicados mediante estructura y similitud, con confirmación humana y métricas de falsos positivos.
- Presentar responsabilidades agregadas por carpetas y ampliar la extracción sintáctica a otros lenguajes.
- Certificar instalaciones limpias, actualizar paquetes y completar revisión de licencias y portabilidad.

Capítulo 8. REFERENCIAS

Referencias numéricas. Los informes de modelos se citan como resultados de sus autores; no son mediciones reproducidas en este trabajo.

[1] DeepSeek-AI, "DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model", arXiv:2405.04434, 2024.

[2] DeepSeek-AI, "DeepSeek-V3 Technical Report", arXiv:2412.19437, 2024.

[3] DeepSeek-AI, "Native Sparse Attention: Hardware-Aligned and Natively Trainable Sparse Attention", arXiv:2502.11089, 2025.

[4] MiniMax, "MiniMax-01: Scaling Foundation Models with Lightning Attention", arXiv:2501.08313, 2025.

[5] MiniMax, "MiniMax-M1", arXiv:2506.13585, 2025.

[6] "Every Attention Matters: An Efficient Hybrid Architecture for Long-Context Reasoning", arXiv:2510.19338, 2025. <https://arxiv.org/abs/2510.19338>

[7] "Scaling Linear Attention with Sparse State Expansion", arXiv:2507.16577, 2025. <https://arxiv.org/abs/2507.16577>

[8] DeepSeek-AI, "DeepSeekMoE: Towards Ultimate Expert Specialization in Mixture-of-Experts Language Models", arXiv:2401.06066, 2024.

[9] "Auxiliary-Loss-Free Load Balancing Strategy for Mixture-of-Experts", arXiv:2408.15664, 2024.

[10] G. Hinton, O. Vinyals, J. Dean, "Distilling the Knowledge in a Neural Network", arXiv:1503.02531, 2015.

[11] Y. Kim, A. M. Rush, "Sequence-Level Knowledge Distillation", arXiv:1606.07947, 2016.

[12a] Y. Gu et al., "MiniLLM: Knowledge Distillation of Large Language Models", arXiv:2306.08543, 2023.

[12b] R. Agarwal et al., "On-Policy Distillation of Language Models (GKD)", arXiv:2306.13649, 2023.

[12c] "Rethinking Kullback-Leibler Divergence in Knowledge Distillation for Large Language Models", arXiv:2404.02657, 2024. <https://arxiv.org/abs/2404.02657>

[13] A. van den Oord, Y. Li, O. Vinyals, "Representation Learning with Contrastive Predictive Coding", arXiv:1807.03748, 2018.

- [14] L. Wang et al., "Improving Text Embeddings with Large Language Models (e5-mistral)", arXiv:2401.00368, 2024.
- [15] "Efficient Code Embeddings from Code Generation Models", arXiv:2508.21290, 2025. <https://arxiv.org/abs/2508.21290>
- [16] A. Kusupati et al., "Matryoshka Representation Learning", arXiv:2205.13147, 2022.
- [17] Qdrant — Vector Database. <https://qdrant.tech>
- [18] Neo4j Graph Database. <https://neo4j.com>
- [19] BAAI, modelos bge-m3 y bge-reranker-v2-m3. <https://huggingface.co/BAAI>
- [20] J. Lin et al., "AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration", arXiv:2306.00978, 2023.
- [21] E. Frantar et al., "GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers", arXiv:2210.17323, 2022.
- [22] ONNX Runtime. <https://onnxruntime.ai>
- [23] B. Hui et al., "Qwen2.5-Coder Technical Report", arXiv:2409.12186, 2024. <https://arxiv.org/abs/2409.12186>
- [24] T. Dettmers, A. Pagnoni, A. Holtzman y L. Zettlemoyer, "QLoRA: Efficient Finetuning of Quantized LLMs", arXiv:2305.14314, 2023. <https://arxiv.org/abs/2305.14314>
- [25] F. Zhang et al., "RepoCoder: Repository-Level Code Completion Through Iterative Retrieval and Generation", arXiv:2303.12570, 2023. <https://arxiv.org/abs/2303.12570>
- [26] W. Liu et al., "GraphCoder: Enhancing Repository-Level Code Completion via Code Context Graph-based Retrieval and Language Model", arXiv:2406.07003, 2024. <https://arxiv.org/abs/2406.07003>
- [27] S. Ouyang et al., "RepoGraph: Enhancing AI Software Engineering with Repository-level Code Graph", arXiv:2410.14684, 2024. <https://arxiv.org/abs/2410.14684>
- [28] ggml-org, "llama.cpp server", documentación y código. <https://github.com/ggml-org/llama.cpp/tree/master/tools/server>
- [29] J. Chen et al., "M3-Embedding: Multi-Linguality, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation", arXiv:2402.03216, 2024. <https://arxiv.org/abs/2402.03216>

Capítulo 9. ANEXOS Y TRAZABILIDAD

- **Anexo A — Estudio de alternativas:** docs/ESTUDIO_RED_OBRERA.md. El estudio se conserva como propuesta inicial con una nota de revisión que aclara sus límites y su relación con la implementación.
- **Anexo B — Contraste entre memoria y código:** docs/CIERRE_TFM_2026-09-10.md, con los límites técnicos y comprobaciones del cierre.
- **Anexo C — Contrato de uso e instalación:** docs/MANUAL.md, docs/GUIA_EVALUACION.md y deploy/LINUX.md. El código de rutas y modelos es la referencia para resolver discrepancias de documentación histórica.
- **Anexo D — Evidencias:** docs/evidencias/2026-09-05/ para las pruebas históricas y docs/evidencias/2026-09-10/ para la revisión actual. Los archivos de cada ejecución indican el alcance; una captura o una simulación no se presenta como ensayo de instalación limpia.
- **Anexo E — Fuentes y artefactos:** revisión Git e inventario de hashes del paquete. URL de entrega: [COMPLETAR]. El Markdown es la fuente de contenido de las exportaciones DOCX y PDF.

Anexo A. Estudio de alternativas de la red obrera

Estudio: la matemática de la red obrera de Quirón

Algoritmos y ecuaciones de los LLM recientes, y cómo reutilizarlos en nuestro worker

TFM Quirón · Anexo A de la memoria · 10 de julio de 2026

Revisión de alcance — 10 de septiembre de 2026. Este documento recoge alternativas estudiadas para el diseño inicial. La versión implementada usa modelos preentrenados descargables (Qwen como generador y BGE-M3 para embeddings);

no se entrenó ni exportó a ONNX el backbone aquí propuesto. Las etiquetas de «verificación adversarial» describen el procedimiento declarado en julio y no sustituyen una reproducción experimental ni una matriz pública de cada juicio. La recuperación actual del índice de código usa archivos y caché local: no es una reconstrucción completa desde el ledger. Los límites y resultados vigentes se recogen en la memoria revisada y docs/CIERRE_TFM_2026-09-10.md.

Este documento extrae **la matemática** de los mecanismos con los que se han construido los LLM recientes —no compara modelos— y explica, para cada uno: **(a)** la ecuación esencial, **(b)** por qué funciona, **(c)** la referencia de arXiv, y **(d)** cómo se aplica a nuestro worker. El objetivo es reutilizar esas matemáticas en **una sola red pequeña, local, destilable y exportable a ONNX/Rust.**

Cómo leer las etiquetas de evidencia

Fue un barrido de fuentes primarias con verificación adversarial (tres pasadas, ~75 afirmaciones sobre >40 fuentes). Cada mecanismo lleva una etiqueta:

- **** Verificado**** — confirmado 3-0 en la verificación adversarial de este barrido.
- **** Fundamental**** — matemática canónica establecida (referencia original); **no re-verificada** adversarialmente aquí, pero es estándar de libro.
- **** Principio de diseño**** — la idea es reutilizable, pero **no la desplegamos tal cual** por fricción con ONNX/Rust o por tamaño.
- **** Cuestión abierta**** — debate activo en la literatura; se declara como tal.

Honestidad metodológica: las cifras de los *technical reports* (–93,3 % de KV-cache, 25 % de FLOPs, «degradación negligible») son **autoreportadas** por sus autores. Las de complejidad ($O(n \cdot d^2)$, tamaño de estado $d \times d$) son propiedades deterministas de la arquitectura. Se distingue donde importa.

0. Marco del diseño inicial

El diseño inicial proponía un daemon asociado al proyecto, con dos proyecciones —vectores en Qdrant y grafo en Neo4j— reconstruibles desde un registro de cambios. Una sola red pequeña habría aportado descripciones y representaciones. Ese diseño no se realizó íntegramente: la versión de cierre usa archivos y caché local, Qwen para describir y BGE-M3 para vectorizar. Las limitaciones de la cadena de hashes se explican en §4.2.1 de la memoria. El worker no ejecuta herramientas, pero una descripción incorrecta sí puede inducir errores en el asistente que la recibe; reconstruir el grafo no valida esa descripción.

Sus tres prioridades ordenan a qué sirve cada matemática de abajo:

(1) aislar por proyecto, (2) enlazar/recuperar vectores a escala, (3) etiquetar qué hace cada archivo sin alucinar.

FAMILIA 1 — Atención eficiente y gestión del KV-cache

El problema: la atención estándar cuesta $O(n^2 \cdot d)$ y su KV-cache crece **lineal** con la longitud. Tres formas de romperlo.

1.1 Multi-head Latent Attention (MLA) — comprimir el KV · Verificado

(a) Ecuación. En vez de cachear K y V por cabeza, se comprimen juntos a un latente de bajo rango y se reconstruyen al vuelo:

$$\begin{aligned} c_t^{KV} &= W^{DKV} \cdot h_t & \text{con } d_c &= d_h \cdot n_h & \text{(down-projection)} \\ k_t^C &= W^{UK} \cdot c_t^{KV} & v_t^C &= W^{UV} \cdot c_t^{KV} & \text{(up-projections)} \\ k_t^R &= \text{RoPE}(W^{KR} \cdot h_t) & & & \text{(clave posicional RoPE, desacoplada y compartida)} \end{aligned}$$

Solo se **cachea** c_t^{KV} (**pequeño**) y la **clave RoPE** k_t^R , no K/V completos.

(b) Por qué funciona. K y V de un token viven en un subespacio de dimensión mucho menor que $d_h \cdot n_h$; una proyección de bajo rango los captura casi sin pérdida. El RoPE se saca aparte porque su rotación **no puede absorberse** en las up-projections (depende de la posición). Resultado: **−93,3 %** de KV-cache respecto a DeepSeek-67B con calidad comparable a MHA.

(c) DeepSeek-V2 arXiv:2405.04434; DeepSeek-V3 arXiv:2412.19437.

(d) Aplicación al worker. Son solo *matmuls* → **exportable a ONNX**. Un KV-cache pequeño abarata el daemon en inferencia autoregresiva en Rust. El truco de absorción de matrices reduce memoria. (*Matiz : la variante con absorción y RoPE desacoplado no es trivial de exportar; el principio low-rank sí.*)

1.2 Native Sparse Attention (NSA) — atención dispersa entrenable · Verificado

(a) Ecuación. Tres ramas combinadas por un *gate* aprendido:

$$o_t^* = \sum_{c \in \{cmp, slc, win\}} g_t^c \cdot \text{Attn}(q_t, \tilde{K}_t^c, \tilde{V}_t^c) \\ g_t^c = \text{sigmoid}(\text{MLP}(x_t)) \in [0, 1]$$

- *cmp*: compresión gruesa de bloques de tokens (contexto global barato).
- *slc*: selección fina de los **top-n bloques** más relevantes.
- *win*: ventana deslizante (contexto local reciente).

Truco clave: los *scores* de selección salen **gratis** de la rama de compresión, $p_t^{\{cmp\}} = \text{softmax}(q_t^T \tilde{K}_t^{\{cmp\}})$, y se reutilizan para rankear los bloques.

(b) Por qué funciona. Es **entrenable end-to-end** (el *gate* es diferenciable), no una poda solo-inferencia; y es *hardware-aligned* (carga bloques KV contiguos para GQA, aprovechando Tensor Cores).

(c) arXiv:2502.11089 (DeepSeek, ACL 2025 best paper).

(d) Aplicación al worker. El **gating diferenciable de ramas** es reutilizable: permite atender selectivamente a los vectores de archivos relacionados dentro de un proyecto **sin coste cuadrático**. La selección top-n sirve a la prioridad 2.

1.3 Atención lineal / lightning + SSE — estado de tamaño fijo · Verificado

(a) Ecuación. El «truco del producto por la derecha» reordena la asociatividad:

$$O = \text{Norm}((Q K^T) V) \rightarrow O = \text{Norm}(Q (K^T V)) \quad O(n^2 \cdot d) \rightarrow O(n \cdot d^2)$$

La asociatividad anterior se refiere al producto sin una no linealidad entre QK^T y V : no permite mover la softmax de la atención convencional. Una formulación causal requiere además su máscara o recurrencia correspondiente.

En forma recurrente con decaimiento, el estado es una matriz **d×d de tamaño fijo**:

$$kv_t = \lambda \cdot kv_{t-1} + k_t^T v_t \quad o_t = q_t \cdot kv_t$$

(b) Por qué funciona. $K^T V$ es un estado $d \times d$ que **no crece con la longitud**: coste **constante por paso** en inferencia. Lo que se pierde es *retrieval*

asociativo (comprimir a estado fijo difumina). Por eso se **hibrida**: 1 bloque de atención completa cada 7 lineales (7:1). **SSE** (arXiv:2507.16577) mitiga la pérdida con una actualización de estado **top-k dura** (filas dispersas) que reduce la interferencia entre informaciones y **desacopla la capacidad de estado del nº de parámetros**.

(c) MiniMax-01 arXiv:2501.08313; MiniMax-M1 arXiv:2506.13585; Ring-linear arXiv:2510.19338; SSE arXiv:2507.16577.

(d) **Aplicación al worker — la pieza más importante de esta familia**. El estado recurrente $d \times d$ es directamente reutilizable y exportable a ONNX: da un **encoder de coste constante por token**, ideal para un daemon que procesa archivos en **streaming** sin re-materializar matrices $n \times n$. Para «enlazar a escala» (prioridad 2) conviene un backbone **mayormente lineal con pocas capas de atención completa**. El top-k duro de SSE ayuda a la prioridad 3 (no mezclar lógicas de archivos distintos → menos alucinación). *(el kernel lightning I/O-aware con tiling no se reproduce fácil en Rust; se toma la forma recurrente, no el kernel.)*

FAMILIA 2 — Mezcla de expertos (MoE) y enrutado

2.1 DeepSeekMoE: experto compartido + expertos de grano fino · Verificado

(a) Ecuación.

$$h_t' = u_t + \sum_{i=1}^{N_s} \text{FFN}_i^{\text{shared}}(u_t) + \sum_{i=1}^{N_r} g_{i,t} \cdot \text{FFN}_i^{\text{routed}}(u_t)$$

$$s_{i,t} = \text{sigmoid}(u_t^T e_i) \quad (\text{afinidad token-experto})$$

$$g_{i,t} = \text{normaliza}(s_{i,t}) \text{ si } i \in \text{TopK}_r, \text{ si no } 0$$

DeepSeek-V3: $N_s=1$ compartido, $N_r=256$ enrutados, $K_r=8$ activos → **671 B** totales, **37 B** activados por token.

(b) **Por qué funciona**. El experto **compartido** captura lo común; los de **grano fino** se especializan. Así se **desacopla la capacidad (parámetros) del cómputo por token**.

(c) DeepSeekMoE arXiv:2401.06066; DeepSeek-V2/V3 arXiv:2405.04434 / 2412.19437.

(d) **Aplicación al worker. Principio de diseño, no despliegue**. Los modelos MoE de la escala citada exceden el equipo disponible; no se concluye que todo MoE sea inviable. Su *routing* requiere estudiar la exportación a ONNX/Rust. El patrón «**una cabeza compartida + cabezas especializadas por tipo de lógica**»

inspira la estructura de **nuestra única red** sin necesidad de *routing* disperso.

2.2 Balanceo de carga SIN pérdida auxiliar · Verificado

(a) Ecuación. Se añade un **sesgo por experto** b_i solo a la **decisión** de enrutado, nunca al peso de mezcla:

```
enruta i ← (s_{i,t} + b_i) ⊞ TopK      (el peso sigue siendo s_{i,t}, sin sesgo)
b_i ← b_i - γ si el experto i está sobrecargado
b_i ← b_i + γ si está infrautilizado   (γ = 0.001)
```

(b) Por qué funciona. La pérdida auxiliar clásica inyecta **gradientes de interferencia** que dañan el objetivo de lenguaje. Aquí el sesgo cambia **qué** experto se elige sin contaminar **cuánto** pesa, así el balanceo **no altera el objetivo**. Eleva el techo de rendimiento.

(c) Loss-Free Balancing arXiv:2408.15664; DeepSeek-V3 arXiv:2412.19437.

(d) Aplicación al worker. Si la red usa varias cabezas, este **control por sesgo** es un *feedback loop* barato para repartir carga **sin término de pérdida extra** — más simple de implementar y de exportar.

FAMILIA 3 — Destilación profesor → estudiante

Hilo conductor: **nuestro profesor se accede por login (Codex/Claude), sin API de pago y sin logits**. La pregunta matemática es: ¿qué destilación es posible si solo tenemos el **texto** que devuelve el profesor?

3.1 KD clásica (Hinton): *soft targets* con temperatura · Fundamental

(a) Ecuación.

$$L = (1 - \alpha) \cdot \text{CE}(y, \sigma(z_s)) + \alpha \cdot T^2 \cdot \text{KL}(\sigma(z_t / T) \parallel \sigma(z_s / T))$$

(b) Por qué funciona. La temperatura T **suaviza** el *softmax* del profesor y expone su «conocimiento oscuro» (qué clases considera parecidas). El factor T^2 compensa que los gradientes del término *soft* escalan como $1/T^2$, para que su magnitud iguale la del término *hard*.

(c) Hinton et al. arXiv:1503.02531. (no re-verificado en este barrido; fórmula canónica.)

(d) Aplicación al worker. ☐ **No aplicable directamente:** exige los **logits** del profesor z_t , que un profesor por login **no expone**.

3.2 Sequence-Level KD — la vía viable sin logits · Verificado

(a) Ecuación. El objetivo exacto suma sobre todas las secuencias (intratable); se aproxima por la **moda** del profesor, hallada con *beam search* $\hat{y}$:

$$L_{\{\text{SEQ-KD}\}} = - \sum_{\{t\}} q(t|s) \cdot \log p(t|s) \approx - \log p_s(t = \hat{y} | s)$$

Es decir: **entrenar al estudiante con cross-entropy sobre el TEXTO que genera el profesor.**

(b) Por qué funciona. Reduce la destilación a *maximum likelihood* sobre las respuestas del profesor. Procedimiento: (1) el profesor genera, (2) recoges su texto, (3) entrenas al estudiante con CE sobre ese corpus. **Solo necesita texto.** (*Matiz honesto: la idea de que «la moda captura casi toda la masa» fue refutada 0-3 en la verificación; el método se sostiene por tratabilidad, no porque esté probado que concentra la probabilidad.*)

(c) Kim & Rush arXiv:1606.07947.

(d) Aplicación al worker. Directamente reutilizable — es nuestra vía. Es exactamente el subcaso *off-policy por respuestas* que un profesor por login sí permite. El estudiante puede ser ~10× más pequeño/rápido con poca pérdida.

3.3 Reverse-KL (MiniLLM) y GKD — por qué NO nos sirven · Verificado

- **MiniLLM** minimiza el **KL inverso** $KL(q_{\text{estudiante}} \square p_{\text{profesor}})$, *mode-seeking* (evita que el estudiante ponga masa donde el profesor casi no la tiene). Su gradiente es un **policy-gradient tipo REINFORCE**:

$$\square_{\theta} L = E_{\{x \square q_s\}} [\square_{\theta} \log q_s(x) \cdot (\log q_s(x) - \log p_t(x))].$$

Es on-policy y necesita p_t token a token → ☐ no viable solo con texto.

arXiv:2306.08543.

- **GKD** generaliza con un objetivo mixto ponderado por λ ($\lambda=0$ off-policy sobre datos fijos; $\lambda=1$ on-policy sobre lo que genera el estudiante), pero **requiere los logits del profesor** sobre esas secuencias.

☐ no viable sin logits. El **único subcaso** que sobrevive sin logits es $\lambda=0$ con *targets* = secuencias del profesor... que **degenera en Sequence-Level KD**.

arXiv:2306.13649.

3.4 Matiz: ¿reverse-KL es realmente mejor? · Cuestión abierta

La dicotomía *mode-seeking* (reverse) vs *mode-averaging* (forward) **presupone** distribuciones continuas y estudiante unimodal. En LLM las distribuciones son **discretas** (*softmax* sobre vocabulario) y q no es unimodal, así que la motivación de MiniLLM está **en disputa**: forward y reverse KL **comparten el mismo óptimo** $q_{\theta} = p$. Conclusión práctica: **no sobre-invertir en reverse-KL**; para un worker destilado solo con respuestas, **Sequence-Level KD (forward implícito por MLE) es lo defendible**. arXiv:2404.02657 (COLING 2025).

FAMILIA 4 — Cuantización y representación

4.1 AWQ — cuantizar protegiendo canales por activación · Verificado

(a) **Ecuación.** Reescalado equivalente por canal (sin *mixed precision*):

$$W_{\times} \approx Q(W \cdot s) \cdot (x / s) \quad (s > 1 \text{ en los canales salientes})$$

$$s^* = \operatorname{argmin}_s \square Q(W \cdot s)(x / s) - W_{\times} \square$$

(b) **Por qué funciona.** El error relativo de redondeo de un canal escala como $1/s$; subir s en los canales **salientes** reduce su error. Y los canales importantes se identifican por la **magnitud de la ACTIVACIÓN**, no del peso (seleccionar por peso rinde casi como al azar). Ej.: OPT-6.7B, proteger 1 % por activación baja la perplejidad $23.54 \rightarrow 11.36$. Es **PTQ**: no reentrena.

(c) AWQ arXiv:2306.00978 (MLSys 2024 best paper).

(d) **Aplicación al worker.** **Directamente reutilizable** para cuantizar a INT4 el backbone del worker y **caber y correr** en 48 GB / producción.

4.2 GPTQ — cuantización PTQ de segundo orden · Verificado

(a) **Ecuación.** Reconstrucción por capa con información del Hessiano (OBS):

$$\operatorname{argmin}_{\{\hat{W}\}} \square W_{\times} - \hat{W}_{\times} \square^2 \quad H_F = 2 \cdot X_F X_F^T$$

$$\delta_F = - (w_q - \operatorname{quant}(w_q)) / [H_F^{-1}]_{\{qq\}} \cdot (H_F^{-1})_{\{:,q\}} \quad (\text{redistribuye el error})$$

(b) **Por qué funciona.** La información de segundo orden **redistribuye el error de redondeo** a los pesos aún no cuantizados. Como en capas grandes el **orden importa poco**, se cuantizan todas las filas en el mismo orden $\rightarrow$ coste $O(d \cdot \text{col}^2)$ en vez de por peso. Cuantiza GPT-175B a 3-4 bits en ~ 4 GPU-h.

(c) GPTQ arXiv:2210.17323 (ICLR 2023).

(d) **Aplicación al worker.** Alternativa/complemento de AWQ para PTQ del backbone.

4.3 InfoNCE — la pérdida que da buenos embeddings · Fundamental

(a) Ecuación.

$$L = - \log \left[\frac{\exp(\text{sim}(q, k_+))}{\sum_i \exp(\text{sim}(q, k_i))} \right]$$

(b) Por qué funciona. Acerca el ancla q a su positivo k_+ y la aleja de los negativos k_i (los demás del *batch*). La temperatura τ controla cuánto se castiga a los negativos difíciles. Es aprendizaje contrastivo estándar.

(c) van den Oord et al. (CPC) arXiv:1807.03748. (*canónico, no re-verificado.*)

(d) Aplicación al worker. Base para entrenar los **embeddings de código 1024-dim** que pueblan Qdrant (prioridad 2). Positivos = pares código $\leftrightarrow$ resumen o código $\leftrightarrow$ código equivalente.

4.4 Last-token pooling — de decoder causal a embedding · Fundamental

(a) Idea. En un *decoder* causal, el **estado del último token** $e = h_L$ resume la secuencia, porque la atención causal deja que ese token «vea» todos los anteriores. Es cómo modelos como e5-mistral / gte-Qwen / jina-code-embeddings producen un vector desde un backbone autoregresivo.

(c) jina-code-embeddings arXiv:2508.21290; e5-mistral arXiv:2401.00368. (*no re-verificado.*)

(d) Aplicación al worker. Permite **reutilizar el mismo backbone destilado** tanto para etiquetar (generar) como para vectorizar (embedding), alineando ambas representaciones. Cuando tocamos un archivo, su vector = h_L de su resumen.

4.5 Matryoshka (MRL) — un vector, muchas dimensiones · Fundamental

(a) Ecuación. Se entrena con la suma de la pérdida sobre **prefijos anidados** del vector:

$$L = \sum_{m \in \{8, 16, 32, \dots, 1024\}} c_m \cdot L_{\text{task}}(z_{\{1:m\}})$$

(b) Por qué funciona. Al forzar que **cada prefijo** $z_{\{1:m\}}$ sea por sí solo un buen embedding, la información se ordena por importancia en las primeras dimensiones. Así el **mismo vector se puede truncar** a 256 o 64 dim **sin reentrenar**, perdiendo poca calidad.

(c) MRL arXiv:2205.13147. (*no re-verificado.*)

(d) Aplicación al worker. Vectores **1024-dim truncables**: búsqueda barata en dimensión baja + reranking en dimensión alta. Escala la prioridad 2 sin duplicar índices.

Síntesis — la red obrera propuesta

Combinando **solo lo reutilizable**, una **única red pequeña** con esta forma:

- **Backbone híbrido mayormente lineal** (estado recurrente $d \times d$, §1.3) con **pocas capas de atención completa** (ratio $\sim 7:1$). El componente recurrente tiene coste constante por paso para dimensiones fijas; el modelo completo conserva la dependencia de la longitud en sus capas de atención completa. Su exportación y rendimiento en ONNX requieren implementación y medición.
- **Una cabeza compartida + cabezas por tipo de lógica** (inspiración del experto compartido de §2.1, *sin routing MoE real*) para etiquetar.
- **Embeddings de código con InfoNCE (§4.3) + last-token pooling (§4.4) + Matryoshka a 1024-dim (§4.5)** → una representación que podría almacenarse en Qdrant. Compartir dimensión no la hace semánticamente compatible con bge-m3; requiere entrenamiento, evaluación y una colección/configuración coherente.
- **Entrenamiento por Sequence-Level KD (§3.2)** desde el profesor por login (solo texto): una vía de destilación viable sin logits, no ejecutada en esta versión.
- **Despliegue cuantizado con AWQ INT4 (§4.1)** para caber en 48 GB y correr en el mismo ort de Rust que ya usa `semantic-ia-local`.

Qué es reutilizable y qué es solo principio de diseño

Mecanismo	Veredicto para el worker
Estado lineal recurrente $d \times d$ (§1.3)	Directamente reutilizable (encoder ONNX)
Selección top-n / gating de NSA (§1.2)	Reutilizable (atender a vectores relacionados)
Sequence-Level KD (§3.2)	Nuestra vía de destilación
AWQ INT4 (§4.1) / GPTQ (§4.2)	Reutilizable (caber y correr)
InfoNCE + last-token + Matryoshka (§4.3–4.5)	Reutilizable (embeddings 1024-dim)
MLA / compresión de KV (§1.1)	Idea sí; export ONNX con fricción
MoE real / <i>routing</i> (§2.1)	Solo el patrón de cabezas; no desplegar

Mecanismo	Veredicto para el worker
Kernel <i>lightning</i> I/O-aware (§1.3)	Forma recurrente sí; kernel no
Reverse-KL / GKD (§3.3)	☐ Necesitan logits del profesor
KD clásica con temperatura (§3.1)	☐ Necesita logits del profesor

Tabla 9. Qué es reutilizable y qué es solo principio de diseño

Cuestiones abiertas y lagunas honestas

- **Sin respaldo primario en este barrido** (fórmulas fundamentales, incluidas por canónicas pero **no re-verificadas** aquí): KD con temperatura T^2 (§3.1), InfoNCE (§4.3), last-token pooling (§4.4), Matryoshka (§4.5). Conviene citarlas como establecidas, no como «verificadas por nosotros».
- forward **vs** reverse **KL** es **debate activo** (§3.4): no cerrar la tesis de MiniLLM como hecho.
- **Fricción real ONNX/Rust** de pesos cuantizados por canal (AWQ INT4) y de MLA/MoE **no está medida** en fuentes primarias: hay que probarlo nosotros.
- **Fidelidad anti-alucinación** de resúmenes *code-to-text* pequeños: no hay evidencia primaria; se medirá con un conjunto etiquetado propio.

Referencias (todas verificadas como fuentes reales)

Atención y KV: MLA — DeepSeek-V2 2405.04434, DeepSeek-V3 2412.19437 · NSA 2502.11089 · Lightning/lineal — MiniMax-01 2501.08313, MiniMax-M1 2506.13585, Ring-linear 2510.19338 · SSE 2507.16577.

MoE: DeepSeekMoE 2401.06066 · Loss-Free Balancing 2408.15664.

Destilación: Hinton 1503.02531 · Sequence-Level KD 1606.07947 · MiniLLM 2306.08543 · GKD 2306.13649 · Forward/Reverse KL 2404.02657.

Cuantización y representación: AWQ 2306.00978 · GPTQ 2210.17323 · InfoNCE 1807.03748 · e5-mistral 2401.00368 · jina-code-embeddings 2508.21290 · Matryoshka 2205.13147.

Síntesis de la propuesta inicial

Se estudiaron **estado lineal recurrente, Sequence-Level KD, AWQ e InfoNCE + last-token pooling + Matryoshka** como componentes de una red futura. La combinación no fue entrenada ni validada en ONNX/Rust. La versión del producto utiliza generador y embeddings preentrenados separados. MLA, MoE y reverse-KL permanecen como alternativas del estudio con las limitaciones descritas sobre el profesor sin logits.